Лабораторная работа 0
Инфраструктура воспроизводимых экспериментов и единый Experiment Harness

Курс “Нейронные сети”

Редакция 2.1 от 9 октября 2026 г.

1 Введение

Курс «Нейронные сети» — введение в изучение нейронных сетей и практическое погружение в них. За семестр вы пройдёте путь от воспроизводимого эксперимента и обучения глубоких сетей к современным моделям. Вы изучите сети для изображений, последовательностей и графов, обучение без разметки, порождающие модели, оценку надёжности предсказаний и объединение нескольких источников данных. Каждая работа посвящена одной теме и устроена одинаково: краткая теория, реализация, небольшой эксперимент и выводы по собственным результатам.

Лабораторная работа 0 является инфраструктурной работой и выполняется до начала основного исследовательского цикла курса. В отличие от последующих работ, ее объектом является не отдельная нейросетевая архитектура, а сам процесс проведения вычислительного эксперимента.

Невоспроизводимость результатов — систематическая проблема машинного обучения [1, 2], а не частный сбой; инфраструктура, устраняющая её причины, и есть предмет ЛР0.

Разработанная в рамках ЛР0 инфраструктура должна использоваться во всех последующих лабораторных работах курса. Предполагается, что новые модели, датасеты, метрики и диагностические процедуры будут добавляться в единый проект, а не создаваться в виде независимых одноразовых ноутбуков.

Основная идея работы:

$$\boxed{\text{Experiment} = \text{Code} + \text{Configuration} + \text{Data} + \text{Randomness} + \text{Environment} + \text{Artifacts}}$$

Результат эксперимента без возможности восстановить перечисленные компоненты не считается полностью воспроизводимым экспериментом.

2 Цель работы

Целью лабораторной работы является разработка единой инфраструктуры для воспроизводимого запуска, регистрации, продолжения, сравнения и профилирования экспериментов с нейронными сетями.

В ходе выполнения работы необходимо не только реализовать соответствующий программный инструментарий, но и экспериментально исследовать границы вычислительной воспроизводимости: влияние случайной инициализации, порядка выборки данных, состояния генераторов случайных чисел и механизма checkpoint/resume.

3 Предполагаемые предварительные знания

Необходимые пререквизиты для выполнениея работы:

- работать с Python и PyTorch;
- создавать класс `torch.nn.Module`;
- использовать `Dataset` и `DataLoader`;
- реализовывать стандартный training loop;
- работать с optimizer и scheduler;
- использовать Git на уровне commit, branch, diff;
- читать и изменять конфигурационные файлы YAML/TOML/JSON.

4 Результаты обучения

После выполнения лабораторной работы сможете:

1. Формализовать вычислительный эксперимент как набор полностью определенных входных условий.
2. Отделять конфигурацию эксперимента от программного кода.
3. Воспроизводимо формировать split по ролям train / val / cal / test и порядок выборки данных.
4. Фиксировать random seeds для Python, NumPy и PyTorch и понимать ограничения такой фиксации.
5. Сохранять происхождение результата: версию кода, конфигурацию, информацию о данных, программном окружении и аппаратной платформе.
6. Корректно сохранять и восстанавливать состояние обучения.
7. Сравнивать несколько запусков одной модели без ручного копирования чисел между notebook-файлами.
8. Выполнять базовое профилирование: число параметров, latency, throughput, время эпохи и использование GPU memory.
9. Оценивать вариативность результата между различными random seeds и определять минимальный эффект, различимый на фоне этой вариативности.
10. Различать воспроизводимость результата и идентичность вычислительной траектории.
11. Различать идентичность эксперимента и идентичность отдельного запуска.
12. Задавать пороги приёмки эксперимента до его проведения и заявлять воспроизводимость только совместно с указанием режима, в котором она достигнута.
13. Планировать вычислительный бюджет серии экспериментов по измеренной стоимости одного прогона.

5 Модель эксперимента

Запуск обучения с номером i полностью определяется пятью составляющими:

$$\mathcal{R}_i = (C_i, \Theta_i, D_i, S_i, V_i)\,, \tag{0.1}$$

где C_i — состояние исходного кода; Θ_i — итоговая конфигурация, то есть настройки после всех переопределений из командной строки; D_i — версия данных и их разбиение на части; S_i — состояние случайности (seed); V_i — окружение: версии библиотек, процессор, видеокарта. Результаты запуска — метрики, журналы, графики, контрольные точки (checkpoints, сохранённые состояния обучения) и затраченное время:

$$A_i = \{M_i,\ L_i,\ P_i,\ K_i,\ T_i\}, \tag{0.2}$$

где M_i — метрики, L_i — журналы, P_i — графики и диагностические данные, K_i — checkpoints, T_i — временные и ресурсные характеристики.

5.1 Два разных идентификатора

Необходимо различать **идентичность эксперимента** и **идентичность запуска**. Это не терминологическая тонкость: смешение этих понятий делает невозможным эксперимент E1.

Идентификатор эксперимента — детерминированная hash-функция от канонического описания условий:

$$EXP_ID_i = H(C_i, \Theta_i, D_i, S_i), \tag{0.3}$$

где H — детерминированная hash-функция (рекомендуется SHA-256) от канонической сериализации аргументов: для C_i — commit hash и признак незакоммиченных изменений, для Θ_i — hash resolved config, для D_i — fingerprint данных и разбиения, для S_i — seed.

Два запуска с одинаковыми кодом, конфигурацией, данными и seed имеют *один и тот же* EXP_ID. Именно это позволяет их сравнивать: совпадение EXP_ID есть утверждение «эти запуски обязаны быть одинаковыми».

Идентификатор запуска дополнительно содержит различитель:

$$RUN_ID_i = EXP_ID_i \,\|\, N_i, \tag{0.4}$$

где $\|$ — конкатенация строк, N_i — монотонный счётчик, метка времени или случайный nonce. Различитель гарантирует, что повторный запуск того же эксперимента не затирает предыдущий.

Практическое следствие. Директория запуска именуется по RUN_ID; группировка, сравнение и поиск выполняются по EXP_ID. Совпадение EXP_ID при различных RUN_ID — нормальная и ожидаемая ситуация, а не ошибка.

Аппаратное окружение V_i не входит в EXP_ID, однако должно быть сохранено в manifest и учитываться при интерпретации воспроизводимости и производительности. Из этого следует, что одинаковый EXP_ID сам по себе не гарантирует одинаковый результат: он гарантирует лишь одинаковость *заявленных* условий.

6 Общая архитектура проекта

Рекомендуемая минимальная структура проекта:

```
project/
|-- configs/
|   |-- lab0.yaml
|   `-- profiles/
|
|-- src/
|   |-- data/
|   |-- models/
|   |-- losses/
|   |-- metrics/
|   |-- engine/
|   |-- diagnostics/
|   |-- utils/
|   |-- train.py
|   |-- evaluate.py
|   |-- compare.py
|   `-- profile.py
|
|-- tests/
|-- runs/
|-- reports/
|-- pyproject.toml
|-- README.md
`-- .gitignore
```

Допускается иная структура проекта при сохранении разделения ответственности между конфигурацией, моделями, данными, циклом обучения, диагностикой и артефактами эксперимента.

Использование Jupyter Notebook допускается для анализа результатов и построения дополнительных визуализаций, но основной эксперимент должен запускаться без ручного выполнения ячеек notebook.

7 Обязательный интерфейс запуска

Минимально должны поддерживаться следующие операции:

```
python -m src.profile_gen --student-id <ID>

python -m src.train --config configs/lab0.yaml

python -m src.evaluate --run <RUN_ID>

python -m src.compare --runs <RUN_ID_1> <RUN_ID_2> ...

python -m src.profile --config configs/lab0.yaml
```

Допускается использование Hydra, Lightning, MLflow, Weights & Biases или других библиотек. Также, все обязательные артефакты должны быть доступны локально.

8 Конфигурация эксперимента

Все параметры, влияющие на эксперимент, должны задаваться через конфигурационный файл или явно регистрируемые command-line overrides.

Пример минимального конфигурационного файла:

```yaml
experiment:
  name: lab0_cifar10
  seed: 1729
  deterministic: true

data:
  name: cifar10
  root: ./data
  train_fraction: 0.25
  num_workers: 0

model:
  name: small_cnn
  width: 64
  depth: 4

optimizer:
  name: adamw
  lr: 0.001
  weight_decay: 0.0001

scheduler:
  name: cosine
  warmup_epochs: 1

train:
  epochs: 10
  batch_size: 128
  amp: false

logging:
  backend: local
  save_every: 1
```

После применения всех overrides должна сохраняться **разрешенная конфигурация** (`resolved config`), то есть конфигурация, фактически использованная при запуске.

Обратите внимание на значения `amp` и `num_workers`. Смешанная точность и параллельная загрузка данных — два типичных источника расхождения между запусками, поэтому в базовой конфигурации они выключены. Сочетание `deterministic: true` с `amp: true` не запрещено, но является отдельным предметом исследования, а не значением по умолчанию.

9 Детерминизм и его границы

Работа посвящена воспроизводимости, поэтому средства, которыми она достигается технически, должны быть названы. Ниже перечислено, что необходимо изучить по документации используемой версии PyTorch [3]; конкретные значения и порядок вызовов подбираются самостоятельно.

9.1 Управление источниками случайности

Как минимум требуется разобраться со следующими механизмами:

- инициализация генераторов `random`, `numpy` и `torch`, включая отдельную инициализацию для CUDA;
- объект `torch.Generator` и его передача в `DataLoader` через аргумент `generator`;
- аргумент `worker_init_fn` и вопрос о том, какое состояние случайности получает каждый worker;
- режим `torch.use_deterministic_algorithms` и его поведение при отсутствии детерминированной реализации операции;
- флаги `deterministic` и `benchmark` в пространстве имён `torch.backends.cudnn`;
- переменная окружения `CUBLAS_WORKSPACE_CONFIG` и условия, при которых она требуется;
- настройки использования TF32 на поддерживающих его устройствах.

9.2 Три режима воспроизводимости

В работе используются три режима. Далее на них ссылаются критерии приёмки экспериментов.

Режим	Условия	Ожидание
A	CPU, `deterministic: true`, `amp: false`, `num_workers: 0`	Побитовая повторяемость достижима
B	CUDA, `deterministic: true`, `amp: false`	Повторяемость с точностью до накопления ошибок округления
C	Любой режим с `amp: true`, `deterministic: false` или `num_workers` > 0	Побитовая повторяемость не гарантируется

Режим A обязателен: все обязательные эксперименты выполняются в нём. Режим B выполняется при наличии CUDA. Режим C исследуется отдельно — именно он показывает, где заканчиваются гарантии.

9.3 Границы гарантий

Фиксация seed не устраняет следующие источники расхождения, и это следует понимать до начала экспериментов:

- недетерминированные реализации отдельных операций на GPU, в частности использующие атомарные операции;

- автоматический подбор алгоритмов свёртки при включённом `benchmark`;
- неассоциативность сложения чисел с плавающей точкой при изменении порядка редукции;
- динамическое масштабирование градиента при смешанной точности;
- смена версии PyTorch, CUDA, cuDNN или драйвера;
- смена модели GPU или числа устройств.

Отсюда основной тезис работы: воспроизводимость есть свойство не программы, а пары «программа и окружение», и заявлять её можно только вместе с указанием режима.

10 Фиксация программного окружения

Manifest обязан содержать версии Python, PyTorch и CUDA, поэтому эти версии должны быть заданы, а не наблюдаемы постфактум.

Минимальные требования: Python не ниже 3.11, PyTorch версии 2.x. Зависимости описываются в `pyproject.toml` или `requirements.txt` с указанием допустимых диапазонов версий. Дополнительно формируется файл блокировки с точными версиями всего разрешённого дерева зависимостей; его hash сохраняется в manifest.

Файл `environment.txt` внутри директории запуска должен позволять восстановить окружение и содержать как минимум: версии Python, PyTorch, CUDA и cuDNN, список установленных пакетов с версиями, название и число устройств, операционную систему.

Различие между диапазоном версий и файлом блокировки существенно. Первый описывает, с чем проект *совместим*; второй — с чем он *фактически был запущен*. Для provenance необходим именно второй.

11 Персонализируемые экспериментальные условия

Experiment Harness должен позволять запускать одну и ту же лабораторную работу с различными индивидуальными профилями без изменения исходного кода.

11.1 Как получить свой профиль

Профиль не выдаётся и не выбирается произвольно, а **вычисляется** из идентификатора студента, поэтому воспроизводим, проверяем и одинаков при каждом вычислении.

Пусть `student_id` — номер зачётной книжки без пробелов. Определим

$$h = \mathrm{SHA256}(\texttt{student_id}), \tag{0.5}$$

и будем обозначать через $h[a{:}b]$ срез его шестнадцатеричного представления (символы с a-го по $(b-1)$-й, нумерация с нуля).

Поля профиля вычисляются так:

Поле	Правило	Множество значений
`profile.id`	`"P"` ‖ h[0:6]	—
`base_seed`	int(h[0:8]) mod 2^{31}	—
`train_fraction`	int(h[8:10]) mod 4	{0.20, 0.25, 0.30, 0.35}
`model.width`	int(h[10:12]) mod 4	{48, 64, 96, 128}
`model.depth`	int(h[12:14]) mod 3	{3, 4, 5}
`train.batch_size`	int(h[14:16]) mod 3	{64, 128, 256}
`optimizer.name`	int(h[16:18]) mod 2	{`adamw`, `sgd`}
`scheduler.name`	int(h[18:20]) mod 2	{`cosine`, `step`}
`profile_batch_sizes`	int(h[20:22]) mod 3	$\{1, 8, 32, 64\}$, $\{1, 8, 32, 128\}$, $\{1, 16, 64, 256\}$

Индексы во множествах отсчитываются с нуля, множества упорядочены так, как записаны. Все целые получаются интерпретацией среза как шестнадцатеричного числа.

Seed для эксперимента E2 порождаются из базового детерминированно, например как `base_seed` + k для $k = 0, \ldots, 4$; выбранное правило фиксируется в отчёте.

Вычисление профиля должно быть реализовано в проекте отдельной командой, чтобы результат не зависел от ручного счёта:

```
python -m src.profile_gen --student-id <ID> --out configs/profiles/<ID>.yaml
```

Результат для `student_id` = `"20250137"` должен иметь вид:

```
profile:
  id: P4f2a1c
  student_id_hash: 4f2a1c...
  base_seed: 1327645489

data:
  train_fraction: 0.30
  num_workers: 0

model:
  width: 96
  depth: 4

train:
  batch_size: 128

optimizer:
  name: sgd

scheduler:
  name: cosine
```

Значения в примере приведены для иллюстрации формата; собственные значения необходимо вычислить.

11.2 Проверяемость

Manifest каждого запуска обязан содержать `profile.id` и `student_id_hash`. Это делает подмену профиля обнаружимой: преподаватель вычисляет ожидаемый профиль по списку группы и сверяет с манифестами.

Запуск, у которого поля профиля в resolved config не соответствуют вычисленным из `student_id_hash`, считается невыполненным. Полный идентификатор студента в репозиторий не попадает — сохраняется только hash.

11.3 Что профиль изменять не может

Профиль задаёт только перечисленные выше поля. Он не изменяет: датасет, архитектуру baseline как таковую, состав обязательных экспериментов, пороги приёмки и режимы A, B, C.

Ключевое требование: **индивидуальный профиль меняет только конфигурацию эксперимента, но не требует изменения training code.**

Проверка этого требования входит в обязательные тесты: запуск с двумя различными профилями не должен требовать ни одного изменения в исходном коде.

12 Бюджет работы и требования к оборудованию

12.1 Число прогонов

Обязательная программа требует не менее 13 полных прогонов обучения в режиме A:

Источник	Прогонов	Примечание
E0. Sanity check	1	
E1. Повторяемость	2	одинаковый EXP_ID
E2. Разброс между seeds	5	один может совпасть с E0
E3. Checkpoint/resume	2	непрерывный и прерванный
Counterfactual, вопрос 2	2	`num_workers` $= 2$
Counterfactual, вопрос 3	1	усечённый checkpoint
Итого	13	

При наличии CUDA к этому добавляются повторы E1 и E3 в режимах B и C — ещё до восьми прогонов. Эксперимент E4 полного обучения не требует.

12.2 Машинное время

Измерьте в E0 время одной эпохи $T_{\text{эп}}$ и оцените полный бюджет как

$$T_{\text{полн}} \approx 13 \cdot K \cdot T_{\text{эп}}, \tag{0.6}$$

где K — число эпох в вашей конфигурации, 13 — число обязательных прогонов из таблицы выше.

Подберите `train_fraction` и `epochs` так, чтобы выполнялось $T_{\text{эп}} \leq 60$ с. При $K = 10$ это даёт около двух с половиной часов машинного времени на обязательную программу. Если на вашем оборудовании условие не выполняется, уменьшайте `epochs` до 5; порог accuracy в E0 при этом должен оставаться достижимым. Значение `train_fraction` задаётся профилем, поэтому его снижение допустимо только как крайняя мера и обязательно фиксируется в отчёте как отклонение от профиля. Выбранные значения и обоснование приводятся в отчёте.

Отсюда практическое следствие, важное для всего курса: параметры baseline подбираются под доступный бюджет до начала экспериментов, а не после того, как выяснилось, что серия не укладывается в срок. Затраченный бюджет (число прогонов, машинное время) — часть отчёта о результате, а не служебная информация [4].

12.3 Выполнение без GPU

При отсутствии CUDA:

- режимы B и C не выполняются, соответствующие столбцы таблиц приёмки не применяются;
- в E4 исключается часть, связанная с памятью GPU; измерения latency и throughput выполняются на CPU и требования к ним сохраняются;
- вопрос 5 counterfactual переформулируется на смену версии PyTorch вместо версии cuDNN.

Существенно, что основное содержание работы при этом не теряется: режим A самодостаточен, а невозможность проверить границы гарантий на GPU фиксируется в отчёте как ограничение исследования.

13 Требования к данным

В качестве базовой задачи рекомендуется использовать CIFAR-10 и компактную CNN. Допускается использование другого небольшого открытого датасета при согласовании, если он позволяет быстро выполнять повторные эксперименты.

В ЛР0 качество модели не является центральным объектом исследования. Главное требование к baseline: модель должна обучаться достаточно быстро, чтобы можно было выполнить серию повторных запусков.

Разбиение данных должно быть воспроизводимым и задаётся split manifest — файлом со списком индексов объектов для каждой роли. В курсе используются четыре роли: *train* (обучение), *val* (выбор модели и гиперпараметров, early stopping; в ЛР8 — подбор температуры), *cal* (калибровочная часть: не читается ни в одной работе до ЛР8, где используется для conformal prediction) и *test* (только итоговые измерения; подбор чего бы то ни было на test запрещён). Для CIFAR-10 разбиение создаётся в ЛР0 один раз и наследуется ЛР2, ЛР4 и ЛР8: из официального train выделяются val (5000 изображений) и cal (2000 изображений), остальное — train; официальный test — роль test. Полный протокол данных курса (включая наборы кадров лиц) — в отдельном документе «Протокол данных курса».

Для каждого эксперимента необходимо сохранять:

- идентификатор и версию датасета;

- размеры частей по ролям train / val / cal / test;
- способ формирования split (стратификация, единица разбиения);
- seed разбиения;
- fingerprint split manifest (hash списков индексов).

Split manifest хранится отдельно от run directory и переиспользуется всеми запусками; его fingerprint входит в D_i и, следовательно, в EXP_ID (0.3).

14 Задание 1. Реализовать Experiment Runner

Необходимо реализовать компонент, который по конфигурации выполняет полный жизненный цикл эксперимента: config → dataset → model → training → evaluation → artifacts.

Runner должен:

1. загрузить и проверить конфигурацию;
2. инициализировать random seeds;
3. сформировать dataset и dataloaders;
4. создать модель;
5. создать optimizer и scheduler;
6. запустить обучение;
7. выполнять validation;
8. сохранять метрики;
9. сохранять checkpoints;
10. зарегистрировать окружение эксперимента;
11. завершить эксперимент с финальным summary.

15 Задание 2. Реализовать Experiment Manifest

Для каждого запуска должен автоматически создаваться файл `manifest.json` или эквивалентный ему структурированный файл.

Минимально он должен содержать:

- EXP_ID и RUN_ID;
- идентификатор профиля и hash идентификатора студента (см. раздел «Персонализируемые экспериментальные условия»);
- дату и время запуска;
- Git commit hash;
- признак наличия незакоммиченных изменений;
- hash конфигурации;
- fingerprint данных и split manifest (роли train / val / cal / test);
- random seed;
- режим детерминизма: значения `deterministic`, `amp`, `num_workers`;
- Python version;
- PyTorch version;
- CUDA version и cuDNN version;
- hash файла блокировки зависимостей;
- тип устройства;

- название GPU/CPU;
- число параметров модели;
- путь к resolved config.

Если рабочее дерево Git содержит незакоммиченные изменения, рекомендуется автоматически сохранять соответствующий `git diff` внутри run directory.

Git history в данной работе рассматривается как часть происхождения (*provenance*) вычислительного эксперимента.

16 Задание 3. Организовать структуру артефактов

Каждый запуск должен иметь собственную директорию:

```
runs/<RUN_ID>/
|-- manifest.json      # contains both EXP_ID and RUN_ID
|-- config.resolved.yaml
|-- metrics.jsonl
|-- summary.json
|-- environment.txt
|-- checkpoints/
|   |-- last.pt
|   `-- best.pt
|-- plots/
`-- git.diff          # only if the working tree is dirty
```

Недопустимо перезаписывать существующую директорию запуска без явного указания пользователя. Повторный запуск с тем же EXP_ID обязан создать новую директорию с новым RUN_ID, а не завершиться ошибкой: именно на этом построен эксперимент E1.

16.1 Что версионируется в Git

Артефакты запусков имеют принципиально разный статус, и политику необходимо задать явно, иначе репозиторий становится непригодным для работы уже после десятка запусков.

Версионируются текстовые артефакты небольшого размера, которые и образуют provenance: `manifest.json`, `config.resolved.yaml`, `metrics.jsonl`, `summary.json`, `environment.txt`, `git.diff` и split manifest.

Не версионируются: `checkpoints/`, `plots/`, загруженные датасеты, кэши. Чекпоинты одного запуска сопоставимы по размеру со всем остальным репозиторием, а восстановить их можно повторным запуском.

Минимальный `.gitignore`:

```
data/
runs/*/checkpoints/
runs/*/plots/
__pycache__/
*.pt
*.pth
```

Существенно, что при таком разделении в репозитории остаётся полная история экспериментов и отсутствуют веса. Отсюда вопрос, на который нужно ответить в отчёте: достаточно ли сохранённого для восстановления результата и что именно теряется вместе с чекпоинтом.

17 Задание 4. Реализовать корректный checkpoint/resume

Checkpoint должен содержать не только веса модели.

Минимальный полный checkpoint:

```
{
    "model": model.state_dict(),
    "optimizer": optimizer.state_dict(),
    "scheduler": scheduler.state_dict(),
    "epoch": epoch,
    "global_step": global_step,
    "python_rng": ...,
    "numpy_rng": ...,
    "torch_rng": ...,
    "cuda_rng": ...
}
```

При использовании AMP также следует сохранять состояние `GradScaler`, если оно присутствует.

После resume обучение должно продолжаться с той же логической позиции, а не начинаться заново с загруженными весами.

18 Задание 5. Реализовать систему логирования

В ходе обучения должны регистрироваться как минимум:

- train loss;
- validation loss;
- основная train metric;
- основная validation metric;
- learning rate;
- epoch;
- global step;
- время эпохи.

Логи должны храниться в машиночитаемом формате.

Допускаются:

- JSONL;
- CSV;
- TensorBoard;
- MLflow;
- Weights & Biases;

- комбинация перечисленных вариантов.

Даже при использовании внешнего сервиса локальный минимальный log должен сохраняться обязательно.

19 Задание 6. Реализовать профилирование модели

Для модели необходимо измерять:

- число trainable parameters;
- среднее время forward pass;
- latency одного batch;
- throughput в samples/second;
- длительность одной training epoch;
- peak GPU memory при использовании CUDA.

Latency необходимо измерять после warm-up.

Для CUDA перед началом и окончанием временного интервала должна выполняться синхронизация устройства.

Рекомендуется сообщать не только среднее значение, но и медиану и один из верхних квантилей, например p_{95}.

20 Задание 7. Реализовать сравнение запусков

Необходимо реализовать инструмент, принимающий несколько RUN_ID и строящий сводную таблицу.

Пример:

RUN_ID	EXP_ID	Seed	Val Acc.	Val Loss	Time	Params
A	X	17	—	—	—	—
B	X	17	—	—	—	—
C	Y	42	—	—	—	—

Запуски A и B имеют одинаковый EXP_ID и потому подлежат сравнению на совпадение; запуск C относится к другому эксперименту, и совпадения от него не требуется.

Инструмент должен уметь определять различия конфигураций между сравниваемыми экспериментами, а также группировать запуски по EXP_ID.

Желательно реализовать функцию вида:

```
diff_runs(run_a, run_b)
```

которая выводит только различающиеся параметры. Для запусков с одинаковым EXP_ID её вывод обязан быть пустым; непустой вывод означает ошибку канонизации.

21 Обязательные эксперименты

21.1 Эксперимент E0. Sanity check

Обучить baseline-модель и убедиться, что:

- loss уменьшается;
- качество превышает случайный baseline;
- метрики корректно записываются;
- checkpoint загружается;
- evaluation выполняется независимо от training script.

21.2 Эксперимент E1. Повторный запуск с одинаковым seed

Выполнить два независимых запуска $\mathcal{R}_A = (C, \Theta, D, S, V)$ и $\mathcal{R}_B = (C, \Theta, D, S, V)$ с одинаковыми компонентами (0.1).

Оба запуска имеют один и тот же EXP_ID и различные RUN_ID. Совпадение EXP_ID является первой проверкой: если оно не достигнуто, канонизация описания эксперимента реализована неверно.

Сравнить:

- состав split по ролям (train / val / cal / test);
- checksum первого batch;
- learning curves;
- финальные метрики;
- при возможности — параметры модели.

Необходимо определить, достигается ли:

1. идентичность данных;
2. идентичность траектории обучения;
3. идентичность финального результата.

Эксперимент выполняется в режиме A обязательно и в режимах B и C при наличии соответствующего оборудования. Если полная идентичность не достигается, требуется локализовать источник расхождения.

21.3 Эксперимент E2. Variance across seeds

При неизменных гиперпараметрах выполнить не менее пяти запусков с различными seed $S \in \{s_1, \ldots, s_N\}$, $N \geq 5$.

Для основной метрики m вычислить среднее и выборочное стандартное отклонение по запускам:

$$\bar{m} = \frac{1}{N} \sum_{i=1}^{N} m_i, \tag{0.7}$$

$$s_m = \sqrt{\frac{1}{N-1} \sum_{i=1}^{N} (m_i - \bar{m})^2}, \tag{0.8}$$

где m_i — значение метрики в запуске с seed s_i, N — число запусков. Результат необходимо представить как минимум в формате $\bar{m} \pm s_m$ с указанием N; в последующих работах та же пара величин используется для всех сравнений по seeds.

Следует сопоставить величину variation между seeds с величиной типичных улучшений, которые могли бы быть ошибочно интерпретированы как эффект новой архитектуры: разброс по seeds регулярно оказывается сравним с заявляемыми в публикациях приростами [5, 6].

21.4 Эксперимент E3. Checkpoint/resume equivalence

Сравнить два режима.

Run A. Непрерывное обучение в течение K эпох.

Run B. Обучение в течение $K/2$ эпох, сохранение полного checkpoint, перезапуск процесса и продолжение до K эпох.

Сравнить относительное расхождение параметров

$$\Delta_\theta = \frac{\|\theta_A - \theta_B\|_2}{\|\theta_A\|_2 + \varepsilon}, \tag{0.9}$$

где θ_A, θ_B — векторы всех параметров моделей после K эпох в режимах A и B, $\varepsilon > 0$ — малая константа (например, 10^{-12}), исключающая деление на нуль; а также финальные метрики и learning curves.

Если траектории расходятся, необходимо определить, какие состояния эксперимента могли быть восстановлены некорректно.

21.5 Эксперимент E4. Производительность

Измерить latency и throughput для нескольких batch size $B \in \{1, 8, 32, 128\}$ или для индивидуально назначенного набора значений.

Построить зависимости latency и throughput от B; при использовании GPU дополнительно — зависимость пикового объёма памяти GPU от B.

Необходимо объяснить, почему batch size, минимизирующий latency одного batch, не обязательно максимизирует throughput.

22 Критерии приёмки экспериментов

Эксперимент, для которого не задан порог, не является экспериментом: его результат нельзя признать ни подтверждающим, ни опровергающим. Ниже пороги заданы явно.

Обозначения: $\varepsilon_{\text{rel}}(x, y) = |x - y| / \max(1, |x|)$ — относительное расхождение; п.п. — процентные пункты.

22.1 E0. Sanity check

- train loss в конце обучения не превышает половины значения после первой эпохи;
- validation accuracy не ниже 30% на CIFAR-10, то есть минимум втрое выше случайного угадывания;

- `metrics.jsonl` содержит запись для каждой эпохи, `summary.json` и `manifest.json` присутствуют;
- `evaluate` по сохранённому checkpoint воспроизводит финальную метрику из `summary.json` с расхождением не более 10^{-6} и запускается отдельно от `train`.

22.2 E1. Повторяемость

Проверяемая величина	Режим A	Режим B	Режим C
Совпадение EXP_ID	точное	точное	точное
Индексы train/val split	точное	точное	точное
Checksum первого batch	точное	точное	не нормируется
ε_{rel} train loss на каждом шаге	0	$\leq 10^{-5}$	не нормируется
Расхождение финальной метрики	0	≤ 0.1 п.п.	не нормируется

Точное совпадение split и checksum обязательно во всех режимах: порядок данных определяется только seed и не зависит от арифметики устройства. Невыполнение этого условия означает ошибку реализации, а не недетерминизм оборудования.

Для режима C порог не задаётся намеренно. Требуется измерить фактическое расхождение и объяснить его источник; отсутствие расхождения в режиме C также требует объяснения.

22.3 E2. Разброс между seeds

- не менее пяти запусков с различными seed при прочих равных;
- приведены $\bar{m}$, s_m и 95% доверительный интервал среднего;
- все запуски имеют различные EXP_ID, различающиеся только компонентой S;
- указана минимальная величина эффекта, которую данная серия способна отличить от шума.

Ориентир для последнего пункта: улучшение меньше $2s_m$ не может считаться установленным без дополнительных запусков. Если полученное s_m превышает 1.5 п.п., требуется указать, чем вызван столь большой разброс.

22.4 E3. Эквивалентность checkpoint/resume

Проверяемая величина	Режим A	Режим B	Режим C
Δ_θ	$\leq 10^{-7}$	$\leq 10^{-5}$	не нормируется
Расхождение финальной метрики	0	≤ 0.1 п.п.	не нормируется
Совпадение числа шагов оптимизатора	точное	точное	точное

Допуск 10^{-7} в режиме A отражает погрешность сериализации, а не недетерминизм вычислений. Совпадение числа шагов обязательно всегда: расхождение означает, что после resume обучение началось не с той логической позиции.

22.5 E4. Производительность

- перед измерением выполнено не менее 10 прогревочных итераций;
- при использовании CUDA выполнена синхронизация устройства до и после измеряемого интервала;
- на каждое значение batch size приходится не менее 30 измерений;
- приведены среднее, медиана и p_{95};
- коэффициент вариации измерений latency не превышает 10%; при большем значении число повторов увеличивается либо объясняется источник нестабильности;
- throughput вычислен как отношение размера batch к времени шага, а не получен пересчётом из среднего latency другого размера batch.

22.6 Отрицательный результат

Отрицательный результат принимается, если источник расхождения локализован и подтверждён отдельным измерением. Например, недостижение порога режима B с указанием конкретной недетерминированной операции является полноценным выполнением эксперимента.

Не принимается: недостижение порога без диагностики, а также подгонка условий эксперимента под порог, например переход в режим A там, где по условию требовался режим B.

23 Уровни воспроизводимости

В рамках работы предлагается различать несколько уровней; различение «воспроизводимости артефакта» и «воспроизводимости вывода» следует [7].

Уровень	Требование
R0	Можно установить происхождение результата: код, config, data fingerprint, environment.
R1	Повторно формируется тот же split и тот же входной поток данных.
R2	Повторный запуск при фиксированном окружении воспроизводит траекторию и итоговые метрики в пределах порогов E1.
R3	Прерванный эксперимент после checkpoint/resume воспроизводит непрерывную траекторию в пределах порогов E3.

Уровни проверяются конкретными экспериментами: R0 — составом manifest, R1 — разделами E1 о split и checksum, R2 — остальными строками E1, R3 — экспериментом E3.

Заявление об уровне без указания режима недопустимо. Корректная формулировка имеет вид «R2 достигнут в режиме A и не достигнут в режиме C»; формулировка «реализация воспроизводима» смысла не имеет.

Студент должен определить, какой максимальный уровень достигается его реализацией и при каких условиях.

24 Counterfactual questions

Ответ на каждый вопрос должен опираться на числа, полученные в собственных запусках, и содержать ссылки на конкретные RUN_ID. Ответ в общем виде, без обращения к собственным результатам, не засчитывается: такие формулировки вынесены в раздел контрольных вопросов.

Часть вопросов требует дополнительного измерения. Это предусмотрено: подобные измерения должны быть дешёвыми, если harness построен правильно.

1. Возьмите из E2 полученные вами $\bar{m}$ и s_m. Какое минимальное улучшение метрики вы способны обнаружить своей серией запусков? Сколько запусков потребовалось бы, чтобы надёжно обнаружить улучшение вдвое меньшее, и укладывается ли это число в бюджет работы?

2. В E1 вы сравнивали два запуска. Повторите сравнение, изменив только значение `num_workers` с 0 на 2. Какие из проверяемых величин перестали совпадать и какие сохранились? Объясните, почему граница прошла именно там.

3. Возьмите checkpoint из E3 и удалите из него всё, кроме `model.state_dict()`. Повторите resume. Приведите новое значение Δ_θ и объясните, какая именно часть состояния отвечает за наблюдаемое расхождение.

4. В ваших запусках E2 найдите два, у которых финальная validation accuracy различается менее чем на 0.1 п.п. Совпадают ли их траектории? Что этот случай говорит о достаточности финальной метрики как свидетельства воспроизведения эксперимента?

5. Возьмите ваш EXP_ID из E1 и укажите, какое поле manifest изменится при переходе на другую версию cuDNN, а какое — нет. Какой уровень из R0–R3 вы сможете гарантировать после такого перехода и почему именно этот?

6. Измените один байт в файле данных, не меняя имени и пути. Что изменится в вашем manifest, а что останется прежним? Обнаружит ли ваш data fingerprint это изменение, и если нет — как его следует изменить?

7. Возьмите ваши измерения из E4 при наименьшем batch size. Сравните latency первой итерации со значением медианы после прогрева. Во сколько раз они различаются и какие процессы происходят в этом интервале?

8. Из E4 возьмите batch size, минимизирующий latency, и batch size, максимизирующий throughput. Совпали ли они? Приведите ваши числа и объясните механизм расхождения либо совпадения.

25 Обязательные тесты

В проекте должны присутствовать автоматические тесты как минимум для следующих свойств:

1. Конфигурация загружается и после resolve содержит все обязательные поля.
2. Один и тот же seed формирует один и тот же split по всем ролям.
3. Checkpoint может быть сохранен и загружен без изменения output модели до продолжения обучения.

4. Метрики на фиксированном наборе входов вычисляются детерминированно.
5. EXP_ID меняется при изменении существенного параметра конфигурации.
6. EXP_ID не меняется при повторном запуске тех же условий, тогда как RUN_ID меняется обязательно.
7. Повторный запуск с существующим EXP_ID создаёт новую директорию и не изменяет содержимое ранее созданных.
8. EXP_ID или manifest отражает изменение состояния кода.
9. Вычисление профиля по одному и тому же `student_id` даёт один и тот же результат, а по различным — различный.
10. Запуск с двумя различными профилями выполняется без изменений в исходном коде.
11. Training script корректно работает хотя бы на CPU; при наличии CUDA устройство выбирается конфигурационно.

26 Что запрещено зашивать в исходный код

Неявные зависимости в коде — один из источников технического долга ML-систем [8]. Следующие параметры не должны быть неявно захардкожены внутри training pipeline:

- абсолютные пути к данным;
- device ID;
- learning rate;
- batch size;
- число эпох;
- seed;
- optimizer;
- scheduler;
- размер модели;
- частота checkpoint;
- split по ролям train / val / cal / test;
- имя директории для конкретного запуска.

Код должен быть переносим между машинами при изменении конфигурации и переменных окружения.

27 Минимальный набор артефактов

По завершении ЛР0 проект должен содержать:

Артефакт	Назначение
Experiment Runner	Единая точка запуска обучения.
Resolved Config	Фактически использованная конфигурация.
Manifest	Происхождение запуска: код, данные, seed, environment.
Checkpoint Manager	Сохранение и восстановление полного состояния обучения.
Metric Logger	Машиночитаемое логирование метрик.

Артефакт	Назначение
Run Comparator	Сравнение результатов и конфигураций нескольких запусков.
Profiler	Latency, throughput, параметры и memory.
Test Suite	Проверка ключевых инвариантов harness.
Profile Generator	Вычисление индивидуального профиля из идентификатора студента.
Dependency Lock	Точные версии всего дерева зависимостей.
Report	Результаты E0–E4 и выводы по воспроизводимости.

28 Структура отчёта

Отчёт — отдельный документ в репозитории. Каждое утверждение о результате сопровождается ссылкой на конкретный RUN_ID; утверждение без такой ссылки проверить невозможно и оно не рассматривается.

1. **Профиль и конфигурация.** Идентификатор профиля, полученные значения полей, выбранные `epochs`, измеренное $T_{\text{эп}}$ и обоснование бюджета. Все отклонения от профиля, если они были.

2. **Устройство harness.** Схема из семи компонентов, состав manifest, состав checkpoint, правило формирования EXP_ID и RUN_ID. Принятые проектные решения и их альтернативы.

3. **Результаты E0–E4.** Для каждого эксперимента: перечень RUN_ID, таблица измеренных величин, сопоставление с порогами приёмки, вывод о выполнении. Для E2 — $\bar{m}$, s_m и доверительный интервал. Для E4 — таблица по batch size с медианой и p_{95}.

4. **Уровни воспроизводимости.** Максимальный достигнутый уровень из R0–R3 с обязательным указанием режима. Для каждого недостигнутого уровня — локализованная причина.

5. **Ответы на counterfactual questions.** Восемь ответов со ссылками на RUN_ID и числами дополнительных измерений.

6. **Ограничения.** Что не проверено и почему: отсутствие GPU, нехватка бюджета, невозможность воспроизвести условие. Известные дефекты собственной реализации.

Объём не нормируется. Нормируется полнота: отсутствие любого из шести разделов делает отчёт непринятым.

29 Расширенная часть

Для желающих расширить систему, предлагаются следующие направления.

29.1 Dataset fingerprinting

Реализовать fingerprint не только для split manifest, но и для версии исходных данных или метаданных.

29.2 Experiment registry

Реализовать локальную базу запусков, поддерживающую запросы вида:

```
find_runs(
    model="small_cnn",
    optimizer="adamw",
    seed=42
)
```

29.3 Automatic config sweep

Добавить механизм grid/random sweep, при котором каждый запуск остается полностью самостоятельным воспроизводимым experiment.

29.4 System monitoring

Логировать:

- GPU utilization;
- CPU utilization;
- RAM;
- GPU memory;
- wall-clock time.

29.5 Experiment lineage

Поддержать связь $\mathcal{R}_{\text{child}} \leftarrow \mathcal{R}_{\text{parent}}$ (ссылку на RUN_ID родительского запуска в manifest), если новый эксперимент является модификацией предыдущего.

Это позволит в последующих лабораторных строить не набор независимых запусков, а историю развития исследовательской гипотезы.

30 Контрольные вопросы

1. Почему фиксированный random seed сам по себе не гарантирует воспроизводимость эксперимента?
2. Чем reproducibility отличается от repeatability в контексте вычислительного эксперимента?
3. Какие состояния необходимо сохранить для корректного checkpoint/resume?
4. Для чего нужен fingerprint данных?
5. Почему Git commit hash недостаточен, если рабочее дерево содержит незакоммиченные изменения?
6. Почему сравнение двух архитектур по одному seed является слабым экспериментальным свидетельством?

7. В чем различие latency и throughput?
8. Почему профилирование GPU требует warm-up и synchronization?
9. Какие параметры эксперимента должны находиться в конфигурации, а какие допустимо оставить внутри кода?
10. Как определить, является ли отличие двух результатов реальным эффектом метода или следствием случайной вариативности?
11. Почему идентификатор эксперимента и идентификатор запуска не могут совпадать?
12. Какие операции на GPU остаются недетерминированными даже при включённом режиме детерминированных алгоритмов и почему?
13. Чем диапазон допустимых версий зависимостей отличается от файла блокировки и какой из них требуется для provenance?

31 Итог лабораторной работы

После выполнения ЛР0 вы должны иметь единый программный каркас, позволяющий представить дальнейшую работу как последовательность воспроизводимых и сравнимых экспериментов.

Главный итог работы — не конкретная accuracy baseline-модели, а способность ответить для любого полученного результата на вопросы:

1. Какой именно код его сформировал?
2. С какой конфигурацией?
3. На каких данных?
4. С каким seed?
5. В каком окружении?
6. Можно ли повторить запуск?
7. Можно ли корректно продолжить его после checkpoint?
8. Насколько результат устойчив к случайной инициализации?
9. Какова вычислительная стоимость эксперимента?

A result without provenance is not a complete experiment.

Литература

[1] O. E. Gundersen, S. Kjensmo. State of the Art: Reproducibility in Artificial Intelligence. *AAAI*, 2018.

[2] J. Pineau, P. Vincent-Lamarre, K. Sinha, V. Larivière, A. Beygelzimer, F. d'Alché-Buc, E. Fox, H. Larochelle. Improving Reproducibility in Machine Learning Research (A Report from the NeurIPS 2019 Reproducibility Program). *JMLR*, 22(164):1–20, 2021.

[3] PyTorch Documentation: Reproducibility. `https://pytorch.org/docs/stable/notes/randomness.html`

[4] J. Dodge, S. Gururangan, D. Card, R. Schwartz, N. A. Smith. Show Your Work: Improved Reporting of Experimental Results. *EMNLP-IJCNLP*, 2019.

[5] P. Henderson, R. Islam, P. Bachman, J. Pineau, D. Precup, D. Meger. Deep Reinforcement Learning that Matters. *AAAI*, 2018.

[6] X. Bouthillier, P. Delaunay, M. Bronzi et al. Accounting for Variance in Machine Learning Benchmarks. *MLSys*, 2021.

[7] X. Bouthillier, C. Laurent, P. Vincent. Unreproducible Research is Reproducible. *ICML*, 2019.

[8] D. Sculley, G. Holt, D. Golovin et al. Hidden Technical Debt in Machine Learning Systems. *NeurIPS*, 2015.